

Software Engineering

Engineering the Development of Software Systems

L05 – Software Engineering in Python – Exercises – Part 2

Marcello Missiroli and Giancarlo Succi
Dipartimento di Informatica – Scienza e Ingegneria
Università di Bologna



Exercises on the OO Paradigm

- Base Objects
- Static stuff
- Abstraction stuff
- Inheritance stuff
- Inheritance & Operators
- Metaclasses, metaclass singleton 1 & 2
- Type Hints
- Introspection & reflection
- Decorator basics, method, class, property
- Abstract classes
- Generics



Base Object

Exercise (p91)

Create the class `Circle`, with the following methods:

- `getArea()`
- `getCircumference()`
- `scale(factor)`

The class should have **no dependencies** and should assume **everything is public by default**.

Write a main program to test its behavior.

Learning Points

Basic class structure, attributes, methods, and encapsulation conventions in Python



Base Object – Solution

Code

```
class Circle:
    def __init__(self, radius):
        self.radius = radius

    def getArea(self):
        return 3.14159 * self.radius ** 2

    def getCircumference(self):
        return 2 * 3.14159 * self.radius

    def scale(self, factor):
        self.radius *= factor
```



Static Stuff

Exercise (p95)

Modify the `Circle` class by adding:

- A class variable `PI` set to `3.14159`
- A class variable `count` that tracks the number of instances created
- A `@classmethod` `getInstanceCount()` that returns how many objects have been instantiated

Update your main program to test the instance count.

Learning Points

Class vs. instance variables, class methods, shared state



Static Stuff – Solution

Code

```
class Circle:
    PI = 3.14159
    count = 0

    def __init__(self, radius):
        self.radius = radius
        Circle.count += 1

    def getArea(self):
        return Circle.PI * self.radius ** 2

    def getCircumference(self):
        return 2 * Circle.PI * self.radius

    def scale(self, factor):
        self.radius *= factor

    @classmethod
    def getInstanceCount(cls):
        return cls.count

c1 = Circle(3) ; c2 = Circle(4)

print("Instances:", Circle.getInstanceCount())
```



Abstraction Stuff

Exercise (p96–98)

Modify the `Circle` class by:

- Declaring `count` as a **protected** class variable `_count`
- Declaring `scale()` as a **protected** method `_scale()`
- Making `radius` a **private** attribute `__radius`
- Try modifying these members directly (should fail), then use name mangling to force the change

Learning Points

Encapsulation, naming conventions, protected vs. private, name mangling



Abstraction Stuff – Solution

Code

```
class Circle: ; PI = 3.14159
    _count = 0 # protected

    def __init__(self, radius):
        self.__radius = radius # private
        Circle._count += 1

    def getArea(self):
        return Circle.PI * self.__radius ** 2

    def getCircumference(self):
        return 2 * Circle.PI * self.__radius

    def _scale(self, factor): # protected
        self.__radius *= factor

    @classmethod
    def getInstanceCount(cls):
        return cls._count
```




Inheritance Stuff

Exercise (p99–101)

Create the subclass `Arc`, derived from `Circle`, with an additional instance variable `radians` (angle in radians).

Instantiate an arc with a radius of 2 and an angle of 90° (i.e., 1.5708 radians), and print its area and perimeter.

Learning Points

Inheritance, use of `super()`, method override, extending class behavior



Inheritance Stuff – Solution

Sample Solution

```
class Arc(Circle):
    def __init__(self, radius, radians):
        super().__init__(radius)
        self.radians = radians

    def getArea(self):
        return (self.radians / (2 * Circle.PI)) * super().getArea()

    def getPerimeter(self):
        return self.radians * self.radius + 2 * self.radius

arc = Arc(2, 1.5708)
print('Area:', arc.getArea())
print('Perimeter:', arc.getPerimeter())
```



Inheritance and Operators

Exercise (p102–106)

Modify the `Circle` class by:

- Overloading the `+` operator to allow summing two `Circle` instances (by radius)
- Converting the `scale()` method into an overload of the `*` operator
- Adding a `__repr__()` method for meaningful string representation

Test the behavior with simple expressions.

Learning Points

Operator overloading, dunder methods, object arithmetic, custom string representation



Inheritance and Operators – Solution

Code

```
class Circle:
    PI = 3.14159

    def __init__(self, radius):
        self.radius = radius

    def getArea(self):
        return Circle.PI * self.radius ** 2

    def getCircumference(self):
        return 2 * Circle.PI * self.radius

    def __add__(self, other):
        if isinstance(other, Circle):
            return Circle(self.radius + other.radius)
        return NotImplemented

    def __mul__(self, factor):
        return Circle(self.radius * factor)

    def __repr__(self):
        return f"Circle(radius={self.radius})"
```



Metaclasses

Exercise (p107–138)

Define a custom metaclass `MetaLogger` that:

- Overrides `__new__`, `__init__`, and `__call__`
- Prints which method is being executed and with what arguments

Then define a class `Figure` using `metaclass=MetaLogger`.

In `Figure`, override `__new__` and `__init__` to print when they are called.

In the `main` block, instantiate `Figure()` and observe the output.

Learning Points

Metaclasses, object lifecycle, overriding construction, introspection



Metaclasses – Solution (Part 1)

Code

```
class MetaLogger(type):
    def __new__(mcs, name, bases, namespace):
        print("MetaLogger.__new__(mcs=%s, name=%s, bases=%s)" % (mcs.__name__, name,
bases))
        return super().__new__(mcs, name, bases, namespace)

    def __init__(cls, name, bases, namespace):
        print("MetaLogger.__init__(cls=%s, name=%s, bases=%s)" % (cls.__name__, name,
bases))
        super().__init__(name, bases, namespace)

    def __call__(cls, *args, **kwargs):
        print("MetaLogger.__call__(cls=%s, args=%s, kwargs=%s)" % (cls.__name__, args,
kwargs))
        return super().__call__(*args, **kwargs)
```



Metaclasses – Solution (Part 2)

Code

```
class Figure(metaclass=MetaLogger):
    def __new__(cls, *args, **kwargs):
        print("Figure.__new__()")
        return super().__new__(cls)

    def __init__(self, *args, **kwargs):
        print("Figure.__init__()")

if __name__ == "__main__":
    print("Creating a Figure...")
    f = Figure()
```



Metaclass Singleton 1

Exercise (p139–142)

Define a metaclass **FigureType** that:

- Keeps a single instance of the **Figure** class using a class-level variable
- Overrides `__new__`, `__init__`, and `__call__` to manage singleton behavior
- Prints diagnostic messages to trace execution

Then define a class **Figure** using **FigureType** as its metaclass:

- In `__new__`, check if the instance already exists
- In `__init__`, initialize with attributes such as `name` and `sides`

In your `main` block:

- Create two **Figure** objects with different parameters
- Show that only one object was created



Metaclass Singleton 1 – Solution (Part 1)

Metaclass Singleton 1 – Solution

```
class FigureType(type):
    _instance = None

    def __new__(cls, name, bases, dct):
        print("__new__ of FigureType")
        return super().__new__(cls, name, bases, dct)

    def __init__(cls, name, bases, dct):
        print("__init__ of FigureType")
        super().__init__(name, bases, dct)

    def __call__(cls, *args, **kwargs):
        print("__call__ of FigureType")
        if FigureType._instance is None:
            print("Creating the unique Figure instance")
            FigureType._instance = cls.__new__(cls, *args, **kwargs)
            FigureType._instance.__init__(*args, **kwargs)
        else:
            print("Returning existing Figure instance")
        return FigureType._instance
```



Metaclass Singleton 1 – Solution (Part 2)

Metaclass Singleton 1 – Solution

```
class Figure(metaclass=FigureType):
    def __new__(cls, *args, **kwargs):
        print("__new__ of Figure")
        if FigureType._instance is None:
            print("Creating the singleton instance of Figure")
            return super().__new__(cls)
        return FigureType._instance

    def __init__(self, name, sides):
        print("__init__ of Figure")
        self.name = name
        self.sides = sides

    def __repr__(self):
        return f"Figure(name={self.name}, sides={self.sides})"
```



Metaclass Singleton 2

Exercise (p143–144)

Solve the previous singleton implementation using a **generic reusable metaclass**.

Define a metaclass `Singleton` that stores a single instance per class in a dictionary.

Then, apply it to the `Figure` class:

- When a new `Figure` is created, check whether an instance already exists
- Print a message indicating if the instance is created or reused
- Prove that it behaves as a singleton

Learning Points

Generic metaclass reuse, centralized singleton logic, scalability across classes



Metaclass Singleton 2 – Solution

Metaclass Singleton 2 – Solution

```
class Singleton(type):
    _instances = {}

    def __call__(cls, *args, **kwargs):
        if cls not in cls._instances:
            print("Creating singleton instance of %s" % cls.__name__)
            cls._instances[cls] = super(Singleton, cls).__call__(*args, **kwargs)
        else:
            print("Returning existing instance of %s" % cls.__name__)
            return cls._instances[cls]

class Figure(metaclass=Singleton):
    def __init__(self, name, sides):
        print("Initializing Figure:  name=%s, sides=%s" % (name, sides))
        self.name = name
        self.sides = sides

    def __repr__(self):
        return "Figure(name=%s, sides=%s)" % (self.name, self.sides)
```



Type Hints

Exercise (p142–152)

Modify the `Circle` class by adding proper **type hints** for:

- Class variables
- Method parameters and return types

Then, run `mypy` to verify type correctness.

Also: include a type mistake in the usage part to test static checking.

Learning Points

Static typing in Python, optional typing, MyPy usage, type safety enforcement



Type Hints – Solution

Code

```
class Circle:
    PI: float = 3.14159

    def __init__(self, radius: float) -> None:
        self.radius: float = radius

    def getArea(self) -> float:
        return Circle.PI * self.radius ** 2

    def getCircumference(self) -> float:
        return 2 * Circle.PI * self.radius

    def __add__(self, other: "Circle") -> "Circle":
        if isinstance(other, Circle):
            return Circle(self.radius + other.radius)
        raise TypeError("Operands must be Circle instances")

    def __mul__(self, factor: float) -> "Circle":
        return Circle(self.radius * factor)

    def __repr__(self) -> str:
        return "Circle(radius=%s)" % self.radius
```



Introspection & Reflection

Exercise (p153–160)

- Create an empty class `Polygon`
- Dynamically add a class attribute `sides = 5`
- Instantiate `p`, and dynamically add `name = "Pentagon"`
- Define a function `describe(self)` that prints radius and radians, then attach it to class `Arc`
- Instantiate `Arc` and call `describe()`
- Using `type()`, create a class `DynamicFigure` with:
 - A class attribute `shape = "hexagon"`
 - A method `describe()` that prints: "This is a hexagon"
- Instantiate `DynamicFigure` and call `describe()`

Learning Points



Introspection & Reflection – Solution (Part 1)

Introspection & Reflection – Solution

```
class Polygon:
    pass

setattr(Polygon, 'sides', 5)

p = Polygon()
setattr(p, 'name', 'Pentagon')
print(Polygon.sides)
print(p.name)

class Arc:
    def __init__(self, radius: float, radians: float):
        self.radius = radius
        self.radians = radians

    def describe(self):
        print("Arc with radius: %s and radians: %s" % (self.radius, self.radians))
        setattr(Arc, 'describe', describe)

a = Arc(2, 1.57)
a.describe()
```




Introspection & Reflection – Solution (Part 2)

Introspection & Reflection – Solution

```
def dynamic_describe(self):  
    print("This is a hexagon")  
  
DynamicFigure = type('DynamicFigure', (), {  
    'shape': 'hexagon',  
    'describe': dynamic_describe })  
  
df = DynamicFigure()  
print(df.shape)  
  
df.describe()
```



Decorator Basics

Exercise (p161–165)

- Define a decorator function `log_call(func)` that:
 - Prints "Calling function: <function name>" before calling the function
 - Prints "Function finished" after the function completes
- Apply the decorator to a function `greet(name)` that prints "Hello, <name>!"
- Call `greet("Alice")`

Learning Points

Function decorators, inner functions, pre/post call logic



Decorator Basics – Solution

Decorator Basics – Solution

```
def log_call(func):
    def wrapper(*args, **kwargs):
        print("Calling function: %s" % func.__name__)
        result = func(*args, **kwargs)
        print("Function finished")
        return result
    return wrapper

@log_call
def greet(name):
    print("Hello, %s!" % name)

greet("Alice")
```



Method Decorators

Exercise (p167)

Define a decorator `timed` that:

- Measures the time a method takes to execute
- Prints the duration in milliseconds

Apply it to the `getArea()` and `getCircumference()` methods of the `Circle` class.

Then:

- Create a `Circle` with a large radius
- Call the methods and observe the timing output

Learning Points

Method decorators, timing functions, non-intrusive instrumentation



Method Decorators – Solution (Part 1)

Method Decorators – Solution

```
import time

def timed(func):
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        end = time.time()
        duration = (end - start) * 1000
        print("%s took %.2f ms" % (func.__name__, duration))
        return result
    return wrapper
```



Method Decorators – Solution (Part 2)

Method Decorators – Solution

```
class Circle:
    PI: float = 3.14159
    def __init__(self, radius: float) -> None:
        self.radius = radius
    @timed
    def getArea(self) -> float:
        return Circle.PI * self.radius ** 2
    @timed
    def getCircumference(self) -> float:
        return 2 * Circle.PI * self.radius

c = Circle(10000)
print(c.getArea())
print(c.getCircumference())
```



Class Decorator

Exercise (p167)

Define a class decorator `tagged(cls)` that:

- Adds a class attribute `tag = "geometry"` to the decorated class
- Returns the modified class

Apply it to the **Figure** class (from earlier exercises).

Then:

- Create a **Figure** instance
- Print the instance and `Figure.tag`

Learning Points

Class decorators, dynamic modification of classes



Class Decorator – Solution

Class Decorator – Solution

```
def tagged(cls):
    cls.tag = "geometry"
    return cls

@tagged
class Figure:
    def __init__(self, name: str, sides: int):
        self.name = name
        self.sides = sides

    def __repr__(self):
        return f"Figure(name={self.name}, sides={self.sides})"

f = Figure("Hexagon", 6)
print(f)

print(Figure.tag)
```




Properties

Exercise (p169–173)

Create a class `CartesianPoint` with:

- Private attributes `__x` and `__y`
- A `__repr__()` method for display
- A protected method `_distanceTo(other)` to compute distance between points
- Properties `x` and `y` to allow controlled read/write access

Test the class by creating 2 Point, modifying coordinates via properties and computing the distance between them

Learning Points

Encapsulation, property decorators, protected methods, controlled attribute access



Properties – Solution (Part 1)

Properties – Solution

```
class CartesianPoint:
    def __init__(self, x, y):
        self.__x = x
        self.__y = y

    @property
    def x(self):
        return self.__x

    @x.setter
    def x(self, value):
        self.__x = value

    @property
    def y(self):
        return self.__y

    @y.setter
    def y(self, value):
        self.__y = value
```



Properties – Solution (Part 2)

Properties – Solution

```
def __repr__(self):
    return "CartesianPoint(x=%s, y=%s)" % (self.__x, self.__y)

def _distanceTo(self, other):
    dx = self.__x - other.x
    dy = self.__y - other.y
    return (dx ** 2 + dy ** 2) ** 0.5

p1 = CartesianPoint(1, 2)
p2 = CartesianPoint(4, 6)
p1.x = 10
print("Modified p1:", p1)
print("Y of p2:", p2.y)
print("Distance:", p1._distanceTo(p2))
```



Abstract Classes

Exercise (p176–180)

Create an abstract class called **Marks**, used to calculate the percentage of marks earned:

- Maths A: 3 subjects out of 100
- Physics: 4 subjects out of 100

The class must contain the abstract method `getPercentage()`, implemented in two subclasses:

- **Math**
- **Physics**

Each subclass stores the student **name** as a property.

Test with:

- Alice in Math: 99, 58, 78
- Bob in Physics: 88, 93, 84, 91



Abstract Classes – Solution (Part 1)

Abstract base and Math subclass

```
from abc import ABC, abstractmethod

class Marks(ABC):
    def __init__(self, name):
        self.name = name
    @abstractmethod
    def getPercentage(self):
        pass

class Math(Marks):
    def __init__(self, name, mark1, mark2, mark3):
        super().__init__(name)
        self.marks = [mark1, mark2, mark3]
    def getPercentage(self):
        return sum(self.marks) / 3
```



Abstract Classes – Solution (Part 2)

Physics subclass and test

```
class Physics(Marks):
    def __init__(self, name, mark1, mark2, mark3, mark4):
        super().__init__(name)
        self.marks = [mark1, mark2, mark3, mark4]
    def getPercentage(self):
        return sum(self.marks) / 4

alice = Math("Alice", 99, 58, 78)
bob = Physics("Bob", 88, 93, 84, 91)
print(f"{alice.name}'s percentage: {alice.getPercentage()}%")
print(f"{bob.name}'s percentage: {bob.getPercentage()}%")
```



Generic

Exercise (p181–186)

Unbound:

Define a generic function `swap(a, b)` using `TypeVar` that returns `(b, a)` for any type `T`.

Bounded:

Create a generic class `ComparableBox[T]` where `T` is bounded to `float` or `int`, with a method `isGreaterThan(other)` that compares internal values.

Learning Points

Generic programming, type variables, type bounds, reusable and type-safe code



Generic – Solution

Unbound

```
from typing import TypeVar, Tuple
T = TypeVar('T')
def swap(a: T, b: T) -> Tuple[T, T]:
    return b, a
print(swap(1, 2))

print(swap("hello", "world"))
```

Bound

```
from typing import TypeVar, Generic
from numbers import Number
T = TypeVar('T', bound=Number)
class ComparableBox(Generic[T]):
    def __init__(self, value: T):
        self.value = value
    def isGreaterThan(self, other: "ComparableBox[T]") -> bool:
        return self.value > other.value
a = ComparableBox(5); b = ComparableBox(3.2)
print(a.isGreaterThan(b)) # True
```


Exercises on language structure

- String interpolation
- IoC & Dependency Injection



IoC & Dependency Injection

Exercise

Create an `OrderProcessor` class that:

- Uses `OrderValidator` to validate orders
- Uses `NotificationService` to send confirmation

Tasks:

- Implement it first without DI (instantiate dependencies internally)
- Refactor to use manual Dependency Injection
- (Optional) Use the `dependency-injector` library for DI

Learning Points

Low coupling, testability, constructor injection, framework-based DI



IoC – No Dependency Injection Solution

Naive implementation (tightly coupled)

```
class OrderValidator:
    def validate(self, order):
        print(f"Validating order: {order}")
        return True

class NotificationService:
    def send_confirmation(self, order):
        print(f"Sending confirmation for order: {order}")

class OrderProcessor:
    def __init__(self):
        self.validator = OrderValidator()
        self.notifier = NotificationService()
    def process(self, order):
        if self.validator.validate(order):
            self.notifier.send_confirmation(order)

processor = OrderProcessor()
processor.process("Order#123")
```



IoC & Dependency Injection Solution

Refactored to accept dependencies externally

```
class OrderProcessor:
    def __init__(self, validator, notifier):
        self.validator = validator
        self.notifier = notifier
    def process(self, order):
        if self.validator.validate(order):
            self.notifier.send_confirmation(order)

validator = OrderValidator()
notifier = NotificationService()
processor = OrderProcessor(validator, notifier)
processor.process("Order#123")
```



Questions?

End of the part two